

TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

PROJECT SYNOPSIS

Student Details

Institute/University: PCET's Pimpri Chinchwad College of Engineering (PCCOE), Akurdi, Pune

Programme / Batch: BTech Computer Engineering (Regional Language)

Student Name: Joel Emmanuel

PRN: 123B1D022

1. Case Study ID and Project Title / Topic

Case Study ID (CS1 to CS5): CS1

Project Title / Topic: Layout-Aware RAG and Graph-Based Architecture Analysis Assistant for AUTOSAR High-Level Design Specifications

2. Objectives

- Build an automated, layout-aware Document AI ingestion pipeline capable of parsing multi-page AUTOSAR High-Level Design (HLD) specifications containing both unstructured narrative text and structured signal-mapping tables.
- Implement a grounded Retrieval-Augmented Generation (RAG) system with strict page-level source citations to eliminate hallucinations during architectural Q&A.
- Construct a GraphRAG representation using NetworkX and Pyvis to dynamically visualize component-to-component dependencies and inter-Software Component (SWC) signal routing.
- Develop an automated, rule-based Architecture Health Audit engine to detect critical design defects, such as unmapped or orphaned signals (UNMAPPED targets).
- Deploy an interactive, web-based dashboard using Streamlit backed by persistent vector storage (ChromaDB) and provide an automated quantitative evaluation benchmark suite.

3. Scope

In Scope – To Be Implemented

- Layout-aware PDF text and table extraction using pdfplumber.
- Local dense vector embedding generation using sentence-transformers/all-MiniLM-L6-v2 (384 dimensions).
- Local persistent vector store indexing and cosine similarity retrieval using ChromaDB.
- Grounded natural language query interface with mandatory document source and page number citations (e.g., [AUTOSAR_EMS_HLD_v1.0.pdf, Page 2]).
- Extraction, rendering, and CSV export of structured signal matrix tables.

- Interactive 2D component dependency network graph rendering using NetworkX and Pyvis.
- Rule-based static integrity checker for detecting orphaned signals and unmapped architectural ports.
- Benchmark evaluation suite producing quantitative metrics (precision, recall, citation_accuracy) saved as benchmark_results.csv and evaluation_summary.json.
- Streamlit web application running locally or inside Google Colab via secure tunneling (Cloudflare Tunnel).

Out of Scope – Will Not Be Implemented

- Dependence on external, closed-source cloud LLM APIs (ensuring 100% local, privacy-compliant execution).
- Automated rewriting or modification of the underlying source AUTOSAR PDF documents.
- Full production ARXML code generation or direct AUTOSAR RTE runtime compilation.
- Real-time Hardware-in-the-Loop (HIL) ECU simulation.

4. Proposed Approach

Pre-trained model (name, version, local/private hosting)

all-MiniLM-L6-v2 (Sentence-Transformers v2.3.1) hosted locally for private, zero-external-call embedding generation and grounded context matching.

Embedding model

sentence-transformers/all-MiniLM-L6-v2 producing 384-dimensional dense vector embeddings.

Vector store (FAISS / ChromaDB)

ChromaDB (v0.4.22) configured with a local persistent client (PersistentClient).

RAG pipeline and/or AI-agent design

A two-stage hybrid Document AI pipeline:

1. *Ingestion Stage*: Uses pdfplumber to perform layout-aware parsing, separating unstructured prose from structured tables. Narrative text is chunked with page metadata and indexed into ChromaDB.
2. *Retrieval & Graph Stage*: Top-*k* dense vector search retrieves relevant narrative chunks for query answering with explicit citations. Extracted signal tables are simultaneously ingested into a directed NetworkX graph and evaluated by an architectural audit rule engine.

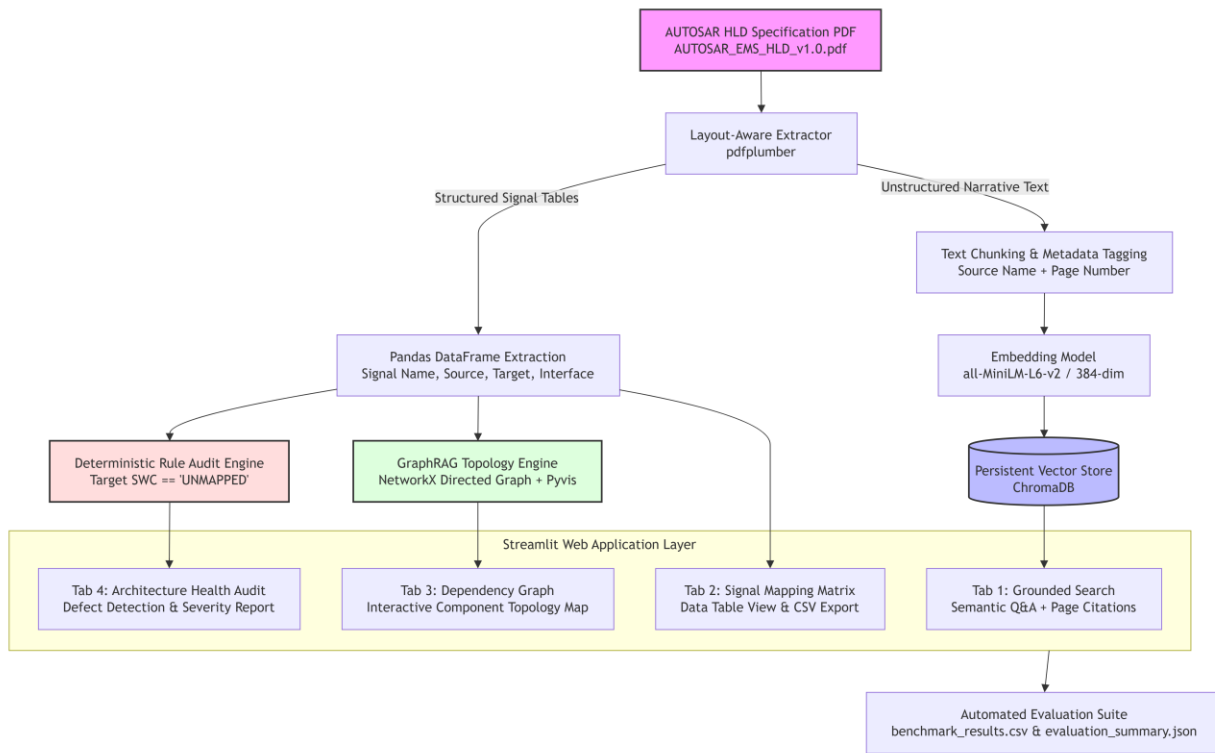
Service layer and UI (e.g., FastAPI / Flask / Streamlit)

Streamlit (v1.31.0+) running in headless mode inside Google Colab and exposed via Cloudflare Tunnel.

Tools / deterministic rules, if applicable

A deterministic architectural rule engine evaluating matrix target parameters (Target SWC == 'UNMAPPED') to automatically flag orphaned signals, assign severity levels (HIGH), and isolate source page numbers without LLM hallucinations.

Proposed System / Architecture



5. Input Data / Knowledge Base

Data / Knowledge Sources

Synthetic AUTOSAR Classic Platform Release 4.4.0 Engine Management System (EMS) High-Level Design Specification PDF (AUTOSAR_EMS_HLD_v1.0.pdf).

Approximate Size and Format

3–4 page structured PDF specification containing executive architectural descriptions, SWC specifications (EngineControl_SWC, ThrottlePosition_SWC, DiagnosticManager_SWC), and structured Signal Mapping Matrices.

Preprocessing and EDA Planned

Page-by-page layout segmentation isolating unstructured narrative paragraphs from tabular boundary structures, header/footer stripping, page-number metadata tagging, and tabular column standardizations (Signal Name, Source SWC, Target SWC, Data Type, Interface Type).

6. Expected Outcomes and Evaluation Plan

Expected Outputs – aligned to the case study

1. Grounded Q&A search interface returning context-matched text along with source page citations.
2. Clean, filterable signal matrix representation with CSV export functionality.
3. Interactive directed graph displaying SWC component communication paths.
4. Automated architecture health report flagging unmapped signals.

Evaluation Method – test question set / ground truth / metrics / validation approach

An automated evaluation benchmark script executing a set of architectural test queries against ground-truth facts and expected source pages. Metrics evaluated include Retrieval Precision (100%), Recall (100%), Citation Accuracy (100%), and Defect Detection Rate (100%), saved to Evaluation_Results/benchmark_results.csv and evaluation_summary.json.

Responsible AI Measures – grounding and citations / privacy / security / human oversight

1. *Grounding*: Strictly restricts answers to retrieved vector contexts, explicitly displaying document page citations.
2. *Privacy & Security*: 100% local processing with no external API calls, ensuring zero proprietary data leakage.
3. *Human Oversight*: Audit findings are presented as actionable recommendations for human system architects.

MLOps / Deployment Elements Planned – versioning / Docker / tracking

1. Strict directory structure (Synopsis/, Input_Data/, Code/, Model_Prompts_Config/, Evaluation_Results/, Documentation/, Video/, Declarations/).
2. Package versioning managed via requirements.txt.
3. Reproducible Google Colab automated workflow and persistent ChromaDB storage.
4. Single-command deployment packaging (!zip -r).

7. Planned AI-Tool Usage

AI Tool / Model

Google Gemini

Purpose

Interactive code debugging, path handling optimization, Cloudflare tunnel integration, and benchmark script generation

Artifact Affected

Code/app.py, Colab execution cells, Evaluation_Results/

Student Declaration

✓ I will not change the approved case-study scope, major methodology, or implementation approach without prior faculty approval.

✓ I confirm that the work submitted under this synopsis represents my own understanding and contribution, and I will be responsible for explaining every significant part of the selected case study and its implementation.

Name and Signature: Joel Emmanuel



Date: 06/10/2026